TaskForge — Day 1 Notes

Python Backend Engineering with FastAPI

Day 1 objective: Set up a Python backend development environment, create a working API, understand HTTP endpoints, and accept validated job submissions.

---

1. What Are We Building?

TaskForge is a background job processing platform.

Applications often need to perform work that shouldn't block a user's request, such as:

- Sending emails
- Generating reports
- Processing files
- Executing webhooks
- Running scheduled tasks

Our long-term goal is to build a system that accepts jobs, stores them, processes them in the background, handles failures, and exposes job status through an API.

Important: At the end of Day 1, we have built the initial API layer. We have not yet implemented persistent storage or actual background job processing.

---

2. Step 1 — Choosing Python and FastAPI

We decided to build TaskForge in Python instead of continuing with Node.js and Express.

Why Python?

Python aligns with our longer-term goals in backend engineering, data science, machine learning, and AI development.

The fundamental backend concepts remain the same regardless of language: HTTP requests, responses, databases, validation, authentication, and error handling.

Why FastAPI?

FastAPI is a Python framework for building APIs.

We chose it because it provides:

- Simple endpoint definitions
- Automatic request validation through Pydantic
- Type hints
- Automatic interactive API documentation
- Support for asynchronous request handling

Our initial technology stack is:

Technology	Purpose
Python	Programming language
FastAPI	API framework
Pydantic	Data validation

Technology	Purpose
Uvicorn	Runs the API server
VS Code	Code editor
Terminal	Commands and testing

---

3. Step 2 — Creating the Project Folder

We created a fresh project folder named `taskforge`.

Our project location:

`/Users/granth/taskforge`

We deliberately started with an empty folder to avoid mixing the old Node.js project with the new Python implementation.

Commands

```
cd ~/taskforge
pwd
ls -la
```

Explanation

- `cd` changes the current directory.
- `~` represents the user's home directory.
- `pwd` prints the current directory.
- `ls -la` lists files, including hidden files.

We confirmed that the folder was empty before setting up the project.

Why this matters: Always know which directory you're working in before creating files, installing dependencies, or running Git commands.

---

4. Step 3 — Creating a Python Virtual Environment

Command

```
python3 -m venv .venv
```

Then we activated it:

```
source .venv/bin/activate
```

What is a virtual environment?

A virtual environment is an isolated Python environment for a particular project.

Suppose you build two projects:

- TaskForge uses FastAPI.
- Another project uses Django.

Separate virtual environments allow each project to maintain its own installed packages and versions.

Without this separation, dependencies from different projects can conflict.

How do we know it is activated?

The terminal prompt begins with something similar to:

```
(.venv) granth@Granths-MacBook-Air taskforge %
```

The `(.venv)` prefix indicates that the virtual environment is active.

Useful commands

Activate the environment:

```
source .venv/bin/activate
```

Check the Python version:

```
python --version
```

Leave the environment:

```
deactivate
```

Remember: You generally need to activate the virtual environment again when opening a new terminal session.

---

5. Step 4 — Installing FastAPI

Commands

```
python -m pip install --upgrade pip
pip install "fastapi[standard]"
```

What did we install?

FastAPI: Handles API routes, requests, validation integration, and responses.

Pydantic: Validates data and converts it into structured Python objects.

Uvicorn: Runs the application as an HTTP server.

The `standard` extra installs FastAPI's recommended additional dependencies, including the server and development tools needed for our setup.

Verify the installation

```
python -m pip show fastapi
```

This displays package information, including the installed version and installation location.

Important lesson

Installing a package and selecting the correct Python interpreter in VS Code are two separate things.

We encountered this warning:

```
Import "fastapi" could not be resolved
```

The problem was that VS Code was using a different Python interpreter from the virtual environment where FastAPI was installed.

How we fixed it

1. Press `Cmd + Shift + P`.
2. Select `Python: Select Interpreter`.
3. Choose the project's interpreter: `/Users/granth/taskforge/.venv/bin/python`
4. Reload VS Code if necessary.

Backend engineering lesson: When imports cannot be resolved, check which Python interpreter is active before reinstalling packages unnecessarily.

---

6. Step 5 — Creating the Project Structure

We created the following files:

```
taskforge/
├── .venv/
└── app/
    ├── __init__.py
    └── main.py
```

What does each file do?

`.venv/`

Contains the project's isolated Python environment and installed packages.

`app/`

Contains the application code.

`__init__.py`

Marks the directory as a regular Python package. It can be empty.

`main.py`

Contains our FastAPI application and endpoint definitions.

Commands

```
mkdir app
touch app/__init__.py
touch app/main.py
```

- `mkdir` creates a directory.
- `touch` creates an empty file if it doesn't already exist.

Why organize code into folders?

As TaskForge grows, we'll separate routes, database operations, schemas, services, and background workers instead of placing everything into one large file.

---

7. Step 6 — Creating the FastAPI Application

Our initial code:

```
from fastapi import FastAPI

app = FastAPI(title="TaskForge API")


@app.get("/")
def home():
    return {
        "message": "Welcome to TaskForge",
        "status": "running",
    }
```

Let's understand every part.

7.1 Importing FastAPI

```
from fastapi import FastAPI
```

This imports the `FastAPI` class from the FastAPI package.

Python's `from ... import ...` syntax lets us import a particular object from a module or package.

7.2 Creating the application

```
app = FastAPI(title="TaskForge API")
```

This creates an instance of the FastAPI class and stores it in the variable `app`.

The `app` object represents our API application.

The title is used in the automatically generated API documentation.

7.3 Understanding decorators

```
@app.get("/")
```

This is a Python decorator used by FastAPI to register a route.

It tells FastAPI that the function immediately below it should handle GET requests to `/`.

For example:

```
@app.get("/")
def home():
    return {"message": "Hello"}
```

When a client sends a GET request to `/`, FastAPI executes `home()`.

7.4 Returning a response

```
return {
    "message": "Welcome to TaskForge",
    "status": "running",
}
```

This is a Python dictionary.

FastAPI automatically converts the returned dictionary into a JSON response.

We don't have to manually construct JSON strings.

What happens when the browser visits `/`?

1. The browser sends an HTTP GET request.
2. FastAPI receives the request.
3. FastAPI matches it to the `/` route.
4. The `home()` function executes.
5. The dictionary is serialized into JSON.
6. The server sends the response to the browser.

This is the basic request-response cycle.

---

8. Step 7 — Creating the Health Endpoint

We added another endpoint:

```
@app.get("/health")
def health_check():
    return {
        "status": "healthy",
        "service": "TaskForge API",
    }
```

What is a health endpoint?

A health endpoint provides a simple way to check whether an application is responding.

Monitoring tools and deployment platforms commonly use health checks to determine whether a service can respond to requests.

Understanding the code

`@app.get("/health")` registers a GET route at `/health`.

`def health_check():` defines the function that handles the request.

`return {...}` returns a dictionary that FastAPI converts into JSON.

Expected response

```
{
  "status": "healthy",
  "service": "TaskForge API"
}
```

A successful request normally returns HTTP status `200 OK`.

Important distinction: This basic endpoint checks whether the application responds. It does not yet verify whether PostgreSQL, Redis, or other dependencies are healthy.

---

9. Step 8 — Running the Backend Server

Command

```
python -m uvicorn app.main:app --reload --port 3001
```

Let's break it down.

Command component	Meaning
`python -m uvicorn`	Runs Uvicorn using the active Python environment

Command component	Meaning
`app.main`	Refers to the `main.py` module inside the `app` package
`:app`	Refers to the FastAPI object named `app`
`--reload`	Restarts the server when code changes during development
`--port 3001`	Uses port 3001 for incoming connections

Why port 3001?

We used port 3001 to avoid a conflict we encountered earlier with another application on port 3000.

Expected output

```
INFO: Uvicorn running on http://127.0.0.1:3001
INFO: Application startup complete.
```

The server must remain running while we test the API.

Important lesson

If the server stops, your browser cannot receive a response from it.

A browser timeout and a Python import warning are different problems:

- An import warning concerns the editor's ability to resolve a package.
- A connection timeout means the browser did not receive a timely response from the server.

Learning to distinguish these problems is an important debugging skill.

---

10. Step 9 — Testing the API

We tested our endpoints through the browser and terminal.

Root endpoint

Open:

`http://127.0.0.1:3001/`

Expected JSON:

```
{
  "message": "Welcome to TaskForge",
  "status": "running"
}
```

Health endpoint

Open:

`http://127.0.0.1:3001/health`

Expected JSON:

```
{
  "status": "healthy",
  "service": "TaskForge API"
}
```

Testing with curl

We can also test the API from a terminal:

```
curl -i http://127.0.0.1:3001/
```

The `-i` flag displays the HTTP response headers along with the response body.

A successful response includes:

```
HTTP/1.1 200 OK
```

What does HTTP 200 mean?

`200 OK` means the HTTP request succeeded.

A status code communicates the outcome of a request to the client.

Some important codes:

Status	Meaning
`200 OK`	Request succeeded
`202 Accepted`	Request accepted for processing
`400 Bad Request`	Request is invalid
`404 Not Found`	Requested route or resource was not found
`422 Unprocessable Entity`	Request data failed validation
`500 Internal Server Error`	Unexpected server-side failure

A `404` does not necessarily mean the server is broken. The server may be running correctly while the requested route does not exist.

---

11. Step 10 — Understanding HTTP Methods

HTTP methods communicate the operation a client wants to perform.

Method	Purpose	TaskForge example
GET	Retrieve information	Retrieve a job's status
POST	Submit or create data	Submit a job
PATCH	Partially update a resource	Update job configuration
DELETE	Delete a resource	Delete a job where permitted

For example:

`GET /api/v1/jobs`

could retrieve a list of jobs.

`POST /api/v1/jobs`

could submit a new job.

The path is the same, but the method represents a different operation.

This is a fundamental API design concept.

---

12. Step 11 — Understanding Pydantic and Request Validation

We then introduced Pydantic to define the structure of a job submission.

Our request model:

```
from typing import Any

from pydantic import BaseModel, Field


class JobCreate(BaseModel):
    type: str = Field(min_length=1)
    payload: dict[str, Any]
```

Let's break this down.

12.1 Importing the required objects

```
from typing import Any
from pydantic import BaseModel, Field
```

`Any` is a Python typing annotation that permits values of different types.

`BaseModel` is the Pydantic base class used to define data models.

`Field` lets us specify additional constraints on fields.

12.2 Creating a request model

```
class JobCreate(BaseModel):
```

This defines a class named `JobCreate` that inherits from `BaseModel`.

It describes what a valid job creation request should contain.

12.3 Validating the job type

```
type: str = Field(min_length=1)
```

This specifies that `type` must be a string with at least one character.

For example:

```
{
  "type": "email"
}
```

has a valid non-empty `type`.

An empty string fails the minimum-length validation.

12.4 Validating the payload

```
payload: dict[str, Any]
```

This specifies that `payload` must be a dictionary with string keys and values that may have different types.

For example:

```
{
  "payload": {
    "to": "user@example.com"
  }
}
```

This is useful because different job types may require different payload fields.

Note: This model checks the outer structure and the non-empty `type`. It does not yet verify that a particular email job contains a valid email address. We'll improve validation as the project develops.

---

13. Step 12 — Building the Job Creation Endpoint

Our endpoint:

```
@app.post(
    "/api/v1/jobs",
    status_code=status.HTTP_202_ACCEPTED,
)
def create_job(job: JobCreate):
    return {
        "message": "Job received successfully",
        "status": "queued",
        "job": job.model_dump(),
    }
```

We also import the status constants:

```
from fastapi import FastAPI, status
```

13.1 Registering the POST endpoint

```
@app.post("/api/v1/jobs", status_code=status.HTTP_202_ACCEPTED)
```

This registers a POST endpoint at `/api/v1/jobs`.

The status code is set to `202 Accepted`.

13.2 Receiving validated data

```
def create_job(job: JobCreate):
```

FastAPI reads the incoming JSON body, validates it against `JobCreate`, and passes the validated data into the function.

If the request is missing required fields or contains invalid data, FastAPI normally returns a `422 Unprocessable Entity` response without executing the endpoint function.

13.3 Returning the response

```
"job": job.model_dump()
```

`model_dump()` converts the Pydantic model into a Python dictionary.

FastAPI then serializes the returned dictionary into JSON.

Important limitation

At this stage, the endpoint only acknowledges the job submission.

It does not:

- Save the job in PostgreSQL.
- Assign a persistent job ID.
- Add the job to a Redis queue.
- Execute the job in the background.

Those capabilities will be introduced in later development stages.

---

14. Step 13 — Testing Through Interactive API Documentation

We opened:

`http://127.0.0.1:3001/docs`

FastAPI generates interactive API documentation automatically.

We can find our endpoints, inspect their request schemas, submit test requests, and examine responses without building a frontend.

Test a valid request

Submit this JSON to `POST /api/v1/jobs`:

```
{
  "type": "email",
  "payload": {
    "to": "user@example.com"
  }
}
```

Expected status:

`202 Accepted`

Expected response body:

```
{
  "message": "Job received successfully",
  "status": "queued",
  "job": {
    "type": "email",
    "payload": {
      "to": "user@example.com"
    }
  }
}
```

Test invalid data

Submit:

```
{
  "type": "",
  "payload": {
    "to": "user@example.com"
  }
}
```

Expected status:

`422 Unprocessable Entity`

The request fails because `type` must contain at least one character.

Why validation matters

Without validation, malformed data can travel deeper into an application and cause unexpected errors.

Validation helps us reject invalid requests early and return useful error responses to clients.

---

15. Complete Day 1 Application

The following is the complete application structure and code we built.

File: `app/main.py`

```
from typing import Any

from fastapi import FastAPI, status
from pydantic import BaseModel, Field

app = FastAPI(title="TaskForge API")


class JobCreate(BaseModel):
    type: str = Field(min_length=1)
    payload: dict[str, Any]


@app.get("/")
def home():
    return {
        "message": "Welcome to TaskForge",
        "status": "running",
    }


@app.get("/health")
def health_check():
    return {
        "status": "healthy",
        "service": "TaskForge API",
    }


@app.post(
    "/api/v1/jobs",
    status_code=status.HTTP_202_ACCEPTED,
)
def create_job(job: JobCreate):
    return {
        "message": "Job received successfully",
        "status": "queued",
        "job": job.model_dump(),
    }
```

Run the server from the project root with your virtual environment activated:

```
python -m uvicorn app.main:app --reload --port 3001
```

Useful URLs:

- Root endpoint: `http://127.0.0.1:3001/`

- Health endpoint: `http://127.0.0.1:3001/health`
- Interactive API docs: `http://127.0.0.1:3001/docs`

---

16. What We Learned on Day 1

By completing Day 1, we practised:

- Creating a Python virtual environment.
- Installing and verifying dependencies.
- Understanding Python imports and packages.
- Creating a FastAPI application.
- Understanding decorators and route registration.
- Defining GET and POST endpoints.
- Understanding HTTP methods and status codes.
- Returning dictionaries as JSON responses.
- Defining request models with Pydantic.
- Validating incoming request bodies.
- Running a server with Uvicorn.
- Testing endpoints in a browser and through interactive API documentation.
- Debugging Python interpreter and server connection issues.

Key concepts to remember

An API endpoint is a route that handles a particular HTTP request.

A request body contains data sent by a client to the server.

A response is the information returned by the server.

Pydantic validation checks whether incoming data conforms to a defined model.

Uvicorn runs our FastAPI application as an HTTP server.

A virtual environment isolates the dependencies of a Python project.

A successful POST response does not automatically mean a job has been stored or executed. We still need persistence and background processing.

---

17. What Remains for TaskForge?

Day 1 establishes the foundation. The main engineering work comes next.

Development stage	Goal
Day 2	Generate unique job IDs and implement an in-memory job lifecycle
Day 3	Improve API structure, error handling, and validation
Next stages	Integrate PostgreSQL and persist jobs
Later stages	Add authentication and authorization
Later stages	Introduce Redis, background workers, retries, and failure handling

Development stage	Goal
Production stage	Add tests, Docker, CI, deployment, monitoring, and benchmarks

This is a suggested progression rather than a promise that each stage will take exactly one day.

Final takeaway: Day 1 was about understanding how a Python API receives a request, validates it, executes a function, and returns a response. That request-response cycle is the foundation of almost every backend application. The next step is to turn our current acknowledgement-only endpoint into a system that actually creates and tracks jobs.